Rapport technique - NexusNexus

Plateforme de visualisation du trafic en temps reel - Frontend Angular et backend Spring Boot

1. Vue d'ensemble

NexusNexus est une plateforme web de visualisation du trafic en temps réel centrée sur Paris. L'objectif principal est d'afficher et de gérer des milliers de véhicules en mouvement sur une carte interactive, tout en conservant une interface fluide, professionnelle et adaptée à un tableau de bord de mobilité intelligente.

Le projet n'est pas conçu comme un jeu vidéo ni comme un simulateur 3D lourd. Il vise plutôt une expérience de supervision opérationnelle : carte sombre, données en temps réel, statistiques, filtres, sélection de véhicules, indicateurs de performance et rendu graphique optimisé.

L'application est divisée en deux grandes parties :

- un frontend Angular responsable de l'interface, de la carte MapLibre et du rendu WebGL des véhicules ;
- un backend Spring Boot responsable de la simulation, du graphe routier, des API REST et du flux WebSocket.

2. Stack technique actuelle

Couche	Technologies
Frontend	Angular 17, TypeScript, SCSS, RxJS
Carte	MapLibre GL JS 4.7, fond de carte sombre de type OpenStreetMap
Rendu véhicules	WebGL2, instancing, custom layers MapLibre
Temps réel	STOMP sur SockJS/WebSocket
Backend	Spring Boot 3, Java 21, Maven
Données routières	OpenStreetMap via Overpass API au démarrage
Visualisations	Heatmap MapLibre, grille de congestion WebGL, trails WebGL, graphiques canvas

3. Fonctionnalités déjà présentes

Le projet contient déjà une base avancée, avec plusieurs fonctionnalités importantes :

- streaming temps réel des véhicules via WebSocket, environ 10 mises à jour par seconde ;
- rendu WebGL2 instancié capable de gérer plusieurs milliers de véhicules ;
- simulation de véhicules sur les routes réelles de Paris grâce aux données OpenStreetMap ;
- routage A* pour que les véhicules suivent des itinéraires réels ;
- interpolation fluide entre les frames reçues du backend ;
- filtres par type de véhicule : voiture, bus, vélo, camion, moto ;
- sélection d'un véhicule sur la carte avec surbrillance et panneau d'information ;

- affichage des trajectoires récentes des véhicules ;
- heatmap de densité de trafic ;
- grille WebGL de congestion ;
- modes de trafic : heure de pointe, normal, nuit ;
- horloge de simulation accélérée ;
- contrôle du nombre de véhicules jusqu'à 5 000 ;
- statistiques de performance : FPS, latence WebSocket, temps de rendu, volume de véhicules ;
- recherche de véhicule avec déplacement automatique de la carte ;
- système d'alertes visuelles ;
- système de replay avec enregistrement de frames ;
- raccourcis clavier pour contrôler l'affichage et la simulation ;
- interface sombre professionnelle.

4. Architecture frontend

Le frontend est organisé autour d'Angular 17 avec des composants standalone. La carte MapLibre occupe l'espace principal et le tableau de bord fournit les contrôles, les statistiques et les états de l'application.

Structure principale :

```
frontend/src/app/
■■■ app.component.ts
■■■ core/
■   ■■■ map-state.service.ts
■   ■■■ selection.service.ts
■   ■■■ replay.service.ts
■   ■■■ alert.service.ts
■   ■■■ chart-data.service.ts
■   ■■■ models/
■■■ map/
■   ■■■ map.component.ts
■   ■■■ heatmap.service.ts
■■■ vehicles/
■   ■■■ vehicle-renderer.ts
■   ■■■ trail-renderer.ts
■   ■■■ congestion-renderer.ts
■   ■■■ spatial-grid.ts
■   ■■■ vehicle-selector.ts
■■■ dashboard/
■   ■■■ dashboard.component.ts
■   ■■■ alerts.component.ts
■   ■■■ charts.component.ts
■   ■■■ search.component.ts
■   ■■■ replay-bar.component.ts
■■■ websocket/
    ■■■ websocket.service.ts
```

Rôle des principales parties frontend

`MapComponent` initialise MapLibre, connecte les couches de rendu et consomme les mises à jour temps réel. Il sert de point central entre la carte, les renderers WebGL et les services applicatifs.

`WebsocketService` gère la connexion STOMP/SockJS avec le backend. Il suit également l'état de connexion et la latence.

`VehicleRenderer` est le coeur du rendu haute performance. Il dessine les véhicules via WebGL2 avec un seul appel instancié plutôt que des éléments DOM.

`TrailRenderer` dessine les lignes de trajectoire récentes des véhicules.

`CongestionRenderer` affiche une grille colorée pour visualiser la densité et la lenteur du trafic.

`SpatialGrid` limite le travail de rendu aux véhicules visibles dans la zone courante de la carte. Cela évite de parcourir inutilement tous les véhicules à chaque frame.

`DashboardComponent` affiche les contrôles, les filtres, les statistiques, les graphiques et les commandes de replay.

`MapStateService` agit comme pont entre la carte et le dashboard. Il évite les dépendances circulaires et centralise les signaux partagés.

5. Architecture backend

Le backend Spring Boot est responsable de la simulation et de la diffusion des données.

Structure principale :

```
backend/src/main/java/com/nexus/
■■■ config/
■   ■■■ SimulationConfig.java
■   ■■■ CorsConfig.java
■   ■■■ WebSocketConfig.java
■■■ controller/
■   ■■■ SimulationController.java
■   ■■■ ReplayController.java
■■■ model/
■   ■■■ Vehicle.java
■   ■■■ VehicleUpdate.java
■   ■■■ VehicleType.java
■   ■■■ SimulationFrame.java
■■■ service/
■   ■■■ FrameRecorder.java
■■■ simulation/
■   ■■■ RoadGraph.java
■   ■■■ RoadNetworkLoader.java
■   ■■■ VehicleGenerator.java
■   ■■■ SimulationEngine.java
■   ■■■ AStarPathfinder.java
■   ■■■ TrafficTimeService.java
■   ■■■ TrafficMode.java
■■■ websocket/
    ■■■ VehicleBroadcaster.java
```

Rôle des principales parties backend

`SimulationConfig` contient la configuration du nombre de véhicules, du rythme de mise à jour et de la zone géographique.

`RoadNetworkLoader` récupère les routes OpenStreetMap de Paris via Overpass API, puis construit un graphe routier.

`RoadGraph` représente les noeuds et les connexions du réseau routier. Il est utilisé par la simulation pour déplacer les véhicules sur des routes réelles.

`AStarPathfinder` calcule des itinéraires entre deux points du graphe routier.

`VehicleGenerator` crée les véhicules initiaux et les positionne sur le réseau.

`SimulationEngine` exécute la logique de mouvement. Il met à jour la position, la vitesse, la direction et la progression des véhicules sur leurs segments.

`VehicleBroadcaster` déclenche les ticks de simulation et diffuse les snapshots vers le frontend via WebSocket.

`SimulationController` expose les commandes REST pour ajuster la simulation.

`ReplayController` expose les commandes liées aux replays.

`FrameRecorder` enregistre les frames pour permettre la lecture historique.

6. Communication frontend-backend

La communication se fait en deux familles :

- REST pour les commandes ponctuelles ;
- WebSocket/STOMP pour les mises à jour temps réel.

Le backend calcule l'état des véhicules à intervalle régulier. Ensuite, il envoie au frontend des messages contenant les positions, vitesses, directions et types des véhicules.

Le frontend ne recrée pas toute l'interface Angular à chaque mise à jour. Les données reçues sont stockées et transmises aux renderers WebGL. Cette séparation est essentielle pour éviter les blocages de l'interface.

Flux simplifié :

```
SimulationEngine
  -> VehicleBroadcaster
  -> WebSocket/STOMP
  -> WebsocketService Angular
  -> MapComponent
  -> SpatialGrid + VehicleRenderer + TrailRenderer + CongestionRenderer
  -> Dashboard stats
```

7. Stratégie de rendu haute performance

Le rendu des véhicules ne repose pas sur des marqueurs HTML, car cette approche ne tiendrait pas avec plusieurs milliers d'objets en mouvement.

Le projet utilise WebGL2 avec rendu instancié :

- les véhicules visibles sont regroupés dans des buffers GPU ;
- un seul appel `drawArraysInstanced` dessine tous les véhicules visibles ;
- chaque instance contient la position projetée, la couleur, la taille, la direction et l'état de sélection ;
- les shaders assurent le rendu visuel avec cercle, glow léger et surbrillance ;
- la taille à l'écran est calculée pour rester stable malgré le zoom.

La grille spatiale limite les calculs aux véhicules dans la zone visible. Le renderer garde aussi une correspondance compacte entre identifiants de véhicules et slots GPU afin d'éviter des trous dans le buffer.

8. Simulation et logique métier

La simulation actuelle est basée sur le réseau routier parisien. Au démarrage, le backend tente de récupérer les données OSM depuis Overpass API. Les routes pertinentes sont transformées en graphe.

Chaque véhicule possède :

- un identifiant ;
- un type ;
- une position ;
- une vitesse ;

- une direction ;
- une route courante ;
- une progression sur le segment ;
- un comportement influencé par le mode de trafic.

Le routage A* permet de choisir des chemins réalistes. Les modes de trafic modifient la vitesse et le comportement global :

- heure de pointe : circulation plus dense et plus lente ;
- normal : comportement standard ;
- nuit : trafic plus fluide.

L'objectif est de garder une logique simple mais scalable. Les calculs critiques évitent les allocations inutiles dans la boucle de tick.

9. Interface utilisateur

L'interface est pensée comme un dashboard de supervision :

- carte sombre plein écran ;
- panneau de contrôle ;
- statistiques temps réel ;
- graphiques canvas ;
- filtres par type de véhicule ;
- sélection et recherche ;
- toggles pour heatmap, trails et congestion ;
- timeline de replay ;
- alertes visuelles.

Le style actuel correspond à un outil professionnel de mobilité intelligente : sombre, clair dans sa hiérarchie, orienté données et sans effets 3D lourds.

10. Configuration actuelle

La configuration backend principale se trouve dans `backend/src/main/resources/application.properties`.

Valeurs importantes :

```
nexus.simulation.vehicle-count=500
nexus.simulation.update-rate-ms=100
nexus.simulation.max-vehicles=5000
nexus.simulation.area.lat-min=48.82
nexus.simulation.area.lat-max=48.90
nexus.simulation.area.lon-min=2.28
nexus.simulation.area.lon-max=2.42
```

Ces valeurs ciblent Paris et permettent de contrôler la charge de simulation.

11. Lancement du projet

Backend :

```
cd backend
./mvnw spring-boot:run
```

Frontend :

```
cd frontend
npm install
npm start
```

Application :

```
http://localhost:4200
```

Le backend écoute normalement sur le port `8080`, et le frontend sur le port `4200`.

12. Points forts actuels

Le projet possède déjà une base technique solide :

- architecture séparée entre rendu, simulation, réseau et UI ;
- rendu véhicules adapté à de gros volumes ;
- utilisation d'un vrai réseau routier OSM ;
- temps réel via WebSocket ;
- contrôles et statistiques déjà présents ;
- capacité de replay ;
- outils de diagnostic de performance ;
- design cohérent avec une plateforme smart mobility.

13. Points à vérifier ou améliorer

Le point explicitement encore à vérifier est la stabilité de la taille des véhicules à tous les niveaux de zoom et en mode carte inclinée.

Les prochaines améliorations naturelles sont :

- ajouter un mode 3D MapLibre avec pitch, bearing et bâtiments extrudés ;
- préparer le support de tuiles vectorielles OSM personnalisées ;
- documenter le pipeline de carte OSM personnalisée ;
- ajouter des couches incidents et géofences ;
- préparer un chargement local du graphe routier en complément d'Overpass ;
- améliorer les contrôles de simulation si certains sont encore absents ;
- tester la montée en charge à 1 000, 5 000 et 10 000 véhicules ;
- envisager un format binaire WebSocket si le JSON devient limitant.

14. Conclusion

NexusNexus est déjà bien plus qu'un prototype simple. Le coeur technique existe : simulation backend, graphe routier OSM, streaming WebSocket, rendu WebGL performant et tableau de bord Angular.

La prochaine phase ne devrait pas repartir de zéro. Elle doit consolider l'existant et ajouter progressivement les fonctionnalités avancées : sensation 3D professionnelle, tuiles OSM personnalisées, incidents, géofences et

meilleures options de configuration.

La priorité doit rester la performance : garder le rendu côté GPU, éviter les éléments DOM pour les véhicules, limiter les allocations et préserver une séparation claire entre interface Angular et boucle de rendu temps réel.